Experiment No. 10
Case Study on Applying reinforcement learning techniques to solve a real- world problem, such as training a self-driving car to navigate a simulated road environment.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim:

To study and analyze the application of reinforcement learning techniques in solving a real-world problem such as autonomous driving in a simulated environment.

Objective:

- **To understand how reinforcement learning is applied in real-world systems**
- **To analyze the working of RL in autonomous vehicles**
- **To explore algorithms used in self-driving cars**
- **To evaluate challenges and performance**

Theory:

1. Reinforcement Learning in Real-World Problems

Reinforcement Learning (RL) is widely used in real-world applications where an agent learns optimal behavior through interaction with an environment using rewards and penalties.

---

2. Problem Statement: Self-Driving Car Navigation

The goal is to train an autonomous vehicle to:

- **Navigate roads**
- **Avoid obstacles**
- **Follow traffic rules**
- **Reach destination safely**

---

3. MDP Formulation for Self-Driving Car

Component	Description
States (S)	Position, speed, surroundings, sensor data
Actions (A)	Accelerate, brake, turn left/right
Reward (R)	+ve for safe driving, -ve for collision
Policy (π)	Driving strategy
Environment	Simulated road (e.g., CARLA, OpenAI Gym)

---

4. RL Algorithms Used

- **Deep Q-Network (DQN) → Discrete actions**
- **Policy Gradient (PPO) → Continuous control**
- **Actor-Critic Methods → Better stability**

---

5. Working of RL in Self-Driving Cars

1. **Car observes environment via sensors (camera, LiDAR)**
2. **State is fed into RL model**
3. **Model predicts best action**
4. **Action is executed (steer, accelerate, brake)**
5. **Reward is given based on performance**
6. **Model updates policy to improve driving**

Implementation (Conceptual Code Example):

```
import gym

import numpy as np



# Simulated environment (simplified)

env = gym.make("CarRacing-v2")



episodes = 10



for ep in range(episodes):

    state = env.reset()[0]

    total_reward = 0



    done = False

    while not done:

        # Random policy (for demo)

        action = env.action_space.sample()



        next_state, reward, done, _, _ = env.step(action)

        total_reward += reward



        state = next_state
```

```
print(f"Episode {ep+1}, Score: {total_reward}")
```

Real-World Example:

Autonomous Driving Systems (e.g., Tesla-like systems)

- **Use deep neural networks + reinforcement learning**
- **Trained on massive driving datasets**
- **Perform tasks like:**
  - **Lane detection**
  - **Object avoidance**
  - **Adaptive cruise control**

Outcome:

- **RL enables vehicles to learn driving behavior without explicit programming**
- **Improves safety and automation**
- **Adapts to dynamic environments**

Challenges:

- **Safety and reliability**
- **High computational cost**
- **Need for large training data**
- **Real-world uncertainty**

- **Ethical concerns (decision-making in accidents)**

Advantages:

- **Learns from experience**
- **Adapts to new environments**
- **Reduces human effort**
- **Handles complex decision-making**

Conclusion:

Reinforcement learning plays a crucial role in autonomous driving by enabling intelligent decision-making through interaction with the environment. While it offers significant advantages in automation and adaptability, challenges like safety, computational complexity, and real-world uncertainty must be addressed for large-scale deployment.